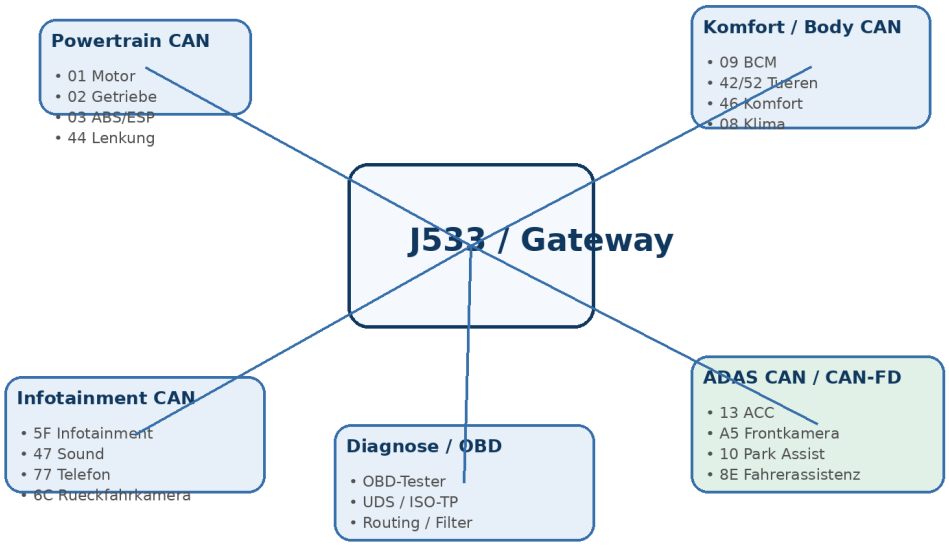
VAG Entwickler-Handbuch

Architektur, Diagnose, UDS, ODX/ASAM, MQB, MIB und ADAS

Deutschsprachige Referenz fuer Entwickler, Diagnose-Tooling und technische Datenbanken

MQB Gateway-Topologie - generische Darstellung



Version: Maerz 2026

Umfang: oeffentlich dokumentierbare Grundlagen, strukturierte Entwicklerperspektive und Datenbank-Blueprints

Sicherheitsrahmen

Dieses Dokument beschreibt Systeme, Standards und Architektur. Es enthaelt keine Anleitungen zur Umgehung von Sicherheitsfunktionen, keine Seed-Key-Verfahren und keine Hinweise zur unautorisierten Manipulation von Fahrzeugen.

Inhaltsuebersicht

1. Zweck, Zielgruppe und Abgrenzung

2. Von PQ zu MQB, MQB Evo und MEB

3. Plattformvergleich auf Entwicklerniveau

3. Elektrik-/Elektronik-Architektur bei VAG

4. Bussysteme und physische Kommunikationsschichten

6. Schichtenmodell fuer Protokolle und Daten

5. Diagnose-Stack: Von OBD bis UDS

8. Benennungskonventionen und Identifikatoren

6. ODX / ASAM und VAG-Datenmodell

7. MQB-Steuergeraete-Map und Adressraum

8. Gateway, Routing und Bus-Topologie

9. UDS-Services fuer VAG-Entwickler

10. DIDs: Was standardisiert ist und was variantenabhaengig bleibt

11. Coding, Anpassung und Variantendefinition

12. Oeffentlich sauber dokumentierbare Coding-Beispiele

13. MIB / Infotainment-Architektur

14. ADAS-Modulstruktur

15. Security, SFD und Diagnose-Restriktionen

16. Praxisorientierter Entwickler-Workflow

17. Referenzarchitektur fuer eine interne Wissensdatenbank

18. Grenzen oeffentlicher Dokumentation

Anhang A: Glossar

Anhang B: Literatur und Quellenhinweise

Wie man dieses Dokument sinnvoll nutzt

Wenn du eine Wissensdatenbank oder ein Tool baust, lies zuerst die Kapitel zu Plattformvergleich, Schichtenmodell, Gateway und UDS. Fuer praktische Tabellen sind vor allem die Kapitel zu Steuergeraete-Map, DIDs, Coding-Kategorien und Datenbankschema relevant.

1. Zweck, Zielgruppe und Abgrenzung

Dieses Handbuch ist als deutschsprachige Entwicklerreferenz fuer die Elektronik- und Diagnosewelt von VAG-Fahrzeugen gedacht. Es richtet sich an Personen, die Diagnose-Stacks, Werkstattsoftware, Logging-Tooling, Testautomatisierung oder technische Datenbanken aufbauen. Der Fokus liegt auf Architektur, Datenmodellen, Kommunikationsschichten, typischen Steuergeraete-Domaenen und sauberer Begriffsklaerung.

Nicht enthalten sind Anleitungen zur Umgehung von Fahrzeugsicherheit, Seed-Key-Angriffe, Immobilizer-Bypaesse oder andere Inhalte, die auf unautorisierte Manipulation zielen. Sicherheitsmechanismen wie SFD oder SFD2 werden nur auf einer abstrahierten, dokumentarischen Ebene beschrieben: was sie bewirken, wie sie den Werkstatt- und Entwicklerprozess veraendern und welche Architekturfolgen daraus entstehen.

Was dieses Dokument bewusst leistet

Es kombiniert oeffentlich dokumentierte Grundlagen zu MQB, UDS, ODX/ASAM, MIB und modernen VAG-Architekturen mit einer Entwicklerperspektive: Welche Schichten existieren, wo liegen die Schnittstellen, wie baut man eine belastbare interne Wissensdatenbank auf und welche Tabellen, IDs und Metadaten sollte man strukturiert verwalten.

2. Von PQ zu MQB, MQB Evo und MEB

Die Bezeichnung einer Plattform ist fuer Entwickler nicht nur Marketing, sondern ein technischer Hinweis auf Paketierung, Bussysteme, Diagnosefaehigkeiten, Wiederverwendung von Steuergeraete-Familien und das erwartbare Sicherheitsmodell. PQ steht fuer aeltere, staerker dezentral organisierte Generationen mit K-Line-Historie und fruehem CAN. MQB brachte die starke Standardisierung quer ueber mehrere Marken und Modellklassen und damit eine deutlich konsistentere Diagnose- und Modullandschaft.

MQB Evo ist keine voellig neue Welt, sondern eine Weiterentwicklung: mehr Security-Mechanismen, zunehmend CAN-FD und Ethernet, mehr Rechenleistung sowie ein staerkerer Fokus auf domainenorientierte bzw. host-zentrierte Architekturanteile. MEB ist als reine BEV-Plattform noch einmal anders zu lesen: Energie-, Lade- und HV-Domaenen ruecken viel staerker in den Vordergrund, waehrend bestimmte klassische Verbrenner-Module entfallen oder in andere Host-Systeme integriert werden.

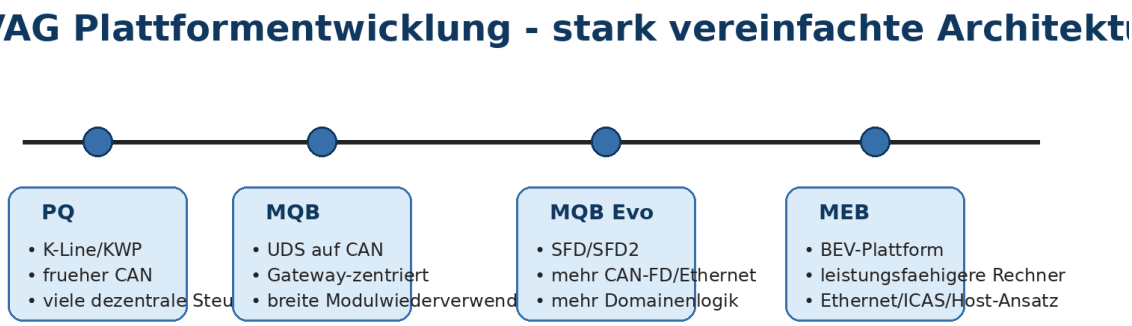


Abbildung 1: Vereinfachte Einordnung der Plattformentwicklung. Die Realitaet ist je nach Marke, Modelljahr und Region deutlich differenzierter.

Fuer Datenbanken und Tools heisst das konkret: Plattform, Modelljahr, Marke, Regionenpaket und teilweise sogar Build-Stand muessen als Schluesselfelder gefuehrt werden. Ein einzelner Eintrag wie 'Adresse 09 = Zentralelektrik' ist ohne Plattformkontext zu grob, weil die interne Funktionszuordnung, Datensaetze, Security und Anpassungskanaele zwischen Generationen erheblich variieren koennen.

3. Plattformvergleich auf Entwicklerniveau

Die folgende Vergleichstabelle ist bewusst hoch verdichtet. Sie ersetzt keine fahrzeugspezifische Analyse, hilft aber bei der ersten Architekturhypothese: Wo ist mit UDS zu rechnen, wo spielen Gateway und ASAM-Datensaetze die groesste Rolle, wo dominieren Security- und Host-Themen?

Merkmal	PQ	MQB	MQB Evo	MEB
Grundcharakter	aeltere, staerker dezentrale Welt	stark standardisierte Querbaukasten-Architektur	Weiterentwicklung mit mehr Security und moderneren Netzanteilen	reine BEV-Plattform mit starker Energie-/Host-Fokussierung
Dominante Diagnosewelt	KWP und frueher CAN-Kontext	UDS auf CAN, ASAM-Datensaetze weithin sichtbar	UDS plus staerkere Security-Restriktionen	UDS, mehr Host-/Gateway-/ICAS-Logik, zunehmende Ethernet-Relevanz
Busbild	K-Line + CAN	CAN, LIN; punktuell weitere Netze	mehr CAN-FD/Ethernet je nach Fahrzeug	stark modernisierte Netze, EV-/HV-Domaenen
Tooling-Fokus	Legacy-Unterstuetzung	breite Werkstatt-/Entwicklerbasis	Capability-Matrix und Restriktionsbehandlung	Domain- und Host-orientierte Sicht
Security-Bild	vergleichsweise offener historischer Kontext	plattform- und ECU-abhaengig	SFD/SFD2 deutlich relevanter	Security und Filter im Alltag staerker praesent
Infotainment-Einordnung	vor MIB/MIB-frueh	MIB stark praesent	MIB3/weitere Integration	starke Online-/Host-Anteile

In echten Projekten sollte diese Grobeinordnung immer mit realen Fahrzeugscans gegengeprueft werden. Besonders bei Uebergangsjahren oder markenspezifischen Derivaten mischen sich Merkmale verschiedener Generationen.

3. Elektrik-/Elektronik-Architektur bei VAG

Die E/E-Architektur laesst sich in mehrere Blickrichtungen zerlegen: physische Steuergeraete, logische Domaenen, Bussysteme, Diagnosepfade und HMI-/Backend-Anbindungen. Aeltere Fahrzeuge haben viele relativ scharf abgegrenzte Steuergeraete mit klarer Zustandsverantwortung. Neuere Fahrzeuge verschieben Teile dieser Logik in leistungsfaehigere Hosts, Gateways oder domainenorientierte Rechner.

Entwickler sollten konsequent zwischen physischem Steuergeraet, logischer Funktion und Diagnoseadresse unterscheiden. Ein 'Modul' im Werkstattjargon kann ein physischer ECU-Knoten, eine softwareseitige Funktionseinheit oder schlicht die Diagnoseadresse sein. In Dokumentationen fuehrt diese Unschaerfe oft zu Fehlern. Empfehlenswert ist daher eine interne Terminologie mit mindestens den Feldern: Plattform, ECU-Name, J-Bezeichnung, Diagnoseadresse, Bus-Domaene, ASAM-Datensatz und Security-Status.

Bei VAG ist der Gateway-Knoten von zentraler Bedeutung. Er bildet den diagnostischen Einstiegspfad, kennt die Installation List bzw. den aktivierten Steuergeraete-Satz und routet Nachrichten zwischen OBD/Tester und den Fahrzeugbussen. In modernen Architekturen kommen Filter, Diagnosefirewalls oder read-only Restriktionen hinzu. Dadurch ist 'OBD-Port vorhanden' nicht gleichbedeutend mit 'voller Zugriff auf alle diagnostischen Funktionen'.

4. Bussysteme und physische Kommunikationsschichten

4.1 K-Line und fruehe Diagnoselandschaft

K-Line und KWP-Generationen praegen die Historie vieler aelterer VAG-Fahrzeuge. Fuer Entwickler ist diese Welt heute vor allem relevant, wenn Legacy-Fahrzeuge, Altgeraete oder historische Retrofit-Projekte betreut werden. Das Design ist deutlich weniger bandbreitenstark als moderne CAN- oder Ethernet-Topologien, aber die Grundidee der dienstorientierten Diagnose ist bereits erkennbar.

4.2 CAN und CAN-FD

CAN ist in VAG-Fahrzeugen die zentrale serielle Backbone-Technologie fuer viele Domaenen. Die praktische Relevanz fuer Tooling ist hoch: Bus-Last, Identifier-Zerlegung, periodische Botschaften, Wake-up-Verhalten, Zustandsabhaengigkeiten und Gateway-Routing bestimmen, was im Mitschnitt sichtbar ist und welche Nachrichten ueberhaupt ankommen. CAN-FD erhoeht Datenrate und Payload-Flexibilitaet, aendert aber nicht die Grundnotwendigkeit, Timing, Session-Status und Gateway-Regeln zu respektieren.

4.3 LIN

LIN ist in VAG-Kontexten haeufig dort anzutreffen, wo einfache Aktoren oder Sensoren kostenguenstig angebunden werden sollen: Spiegel, Fensterheber-Teilfunktionen, Sitzmodule, Schalter, Klima-Peripherie. Wichtig fuer Diagnose- und Datenbankdesign: Nicht jeder LIN-Knoten ist direkt per OBD sichtbar; haeufig laeuft die Diagnose ueber einen Master wie BCM, Tuersteuergeraet oder Klimasteuerung.

4.4 Automotive Ethernet

Mit MIB-Generationen, Kamera- und Assistenzsystemen sowie modernen EV-/Host-Architekturen gewinnt Ethernet an Bedeutung. Fuer Entwickler bedeutet das vor allem: hoehere Bandbreiten, neue Diagnosepfade wie DoIP, staerkere Trennung von klassischem CAN-Coding und softwareorientierten Update-/Backend-Funktionen. Wer neue Tools baut, sollte CAN und Ethernet nie als Gegensaetze sehen, sondern als koexistierende Schichten im selben Fahrzeug.

6. Schichtenmodell fuer Protokolle und Daten

Viele Missverstaendnisse entstehen, weil Protokollschichten und Funktionsbegriffe vermischt werden. 'UDS' ist nicht das gleiche wie 'CAN', 'ODX' nicht das gleiche wie 'DID' und 'Coding' nicht das gleiche wie 'WriteDataByIdentifier'. Das folgende Schichtenmodell ist fuer Architekturgespraeche oft hilfreicher als lange Textlisten.

Schicht	Typische Standards/Beispiele	Entwicklerrelevanz
Physik	K-Line, CAN, CAN-FD, Ethernet, LIN	Bestimmt Interface, Timing, Payload und Mitschnittform
Transport	ISO-TP auf CAN, DoIP auf IP	Segmentierung, Sessionstabilitaet, Error-Handling
Diagnose-Service	UDS, historisch KWP	Servicekatalog, Sessions, NRCs
Datenmodell	ODX/ASAM, DIDs, Routinen, DTCs	Semantik der Datenobjekte
Funktionsebene	Coding, Adaptation, Basic Settings, Flashing	Nutzer- und Werkstattfunktionen

Mit diesem Modell lassen sich auch Projektgrenzen besser ziehen: Wer ein CAN-Logger-Tool baut, arbeitet zunaechst auf Physik- und Transportebene. Wer eine Werkstattanwendung baut, braucht darueber hinaus Service- und Datenmodellebene. Wer eine Wissensdatenbank bauen will, modelliert schliesslich noch Funktionsebenen und Variantenlogik.

5. Diagnose-Stack: Von OBD bis UDS

Ein robustes Verstaendnis des Diagnose-Stacks verhindert viele Fehlannahmen. Die Schichten lassen sich vereinfacht so lesen: physischer Transport (K-Line, CAN, Ethernet), gegebenenfalls Segmentierung/Transport (ISO-TP auf CAN), Diagnoseservices (z. B. UDS) und darauf aufbauend OEM-spezifische Datenmodelle wie DIDs, Routinen, Anpassungen oder Codierungen. Genau an dieser Stelle wird ODX/ASAM fuer Werkstatt- und Entwicklungstools entscheidend.

UDS ist heute der dominante Layer-7-Baustein. Die Services selbst sind standardisiert, die Bedeutung vieler Datenobjekte jedoch nicht automatisch. Ein Request auf Service 0x22 sagt zunaechst nur: 'Read Data by Identifier'. Was hinter einem bestimmten DID steht, haengt von OEM, ECU-Familie und Softwarestand ab. Deshalb muessen Datenbanken nicht nur UDS-SIDs, sondern auch ODX/ASAM-Datensaetze, Varianten und gegebenenfalls ECU-Softwarestaende modellieren.

Praktische Entwicklerregel

Ein Diagnose-Frame ohne Kontext ist nur ein Byte-String. Erst mit Session-Status, Security-Level, Datenmodell (ODX/ASAM), Variantendefinition und ECU-Identitaet wird daraus belastbare Information.

8. Benennungskonventionen und Identifikatoren

In VAG-Projekten sollte man sich frueh auf konsistente Bezeichner einigen. Sonst sprechen Team A und Team B ueber dasselbe Steuergeraet mit drei unterschiedlichen Namen. Besonders problematisch wird das, wenn Diagnoseadresse, J-Nummer, Marketingname, Komponententext und interne Projektbezeichnung parallel existieren.

Feld	Beispiel	Warum es wichtig ist
Marke	VW / Audi / Seat / Skoda / Cupra	Gleiche Plattform, aber andere Softwarestaende und HMI-Logiken moeglich
Plattform	MQB / MQB Evo / MEB	Bestimmt Grundannahmen zu Architektur und Security
Adresse	19	Diagnoseadresse, nicht gleich physische Komplettfunktion
J-Bezeichnung	J533	Werkstatt-/Schaltplannahe ECU-Bezeichnung
ASAM Dataset	EV_Gatew...	Schluessel fuer moderne ECU-Identitaet
SW-Nummer	5Q0 907 ...	Versionierung und Variantenzuordnung
Komponente	ICAS1 Host-SG ...	Hilft bei Generationseinordnung
VCID / Scan-Metadaten	toolabhaengig	Wichtig fuer Wiederholbarkeit und Vergleich

Eine praxistaugliche Regel lautet: die Diagnoseadresse als primaeren Kurznamen fuehren, J-Bezeichnung und ASAM-Datensatz als technische Schluessel speichern und Marketing-/Werkstattbezeichnungen nur als Anzeigealias verwenden.

6. ODX / ASAM und VAG-Datenmodell

ODX (ASAM MCD-2 D) ist fuer die Entwicklerperspektive zentral, weil hieraus die formale Beschreibung der Diagnosekommunikation stammt: Requests, Responses, DTCs, Parameter, Routinen, Coding-Informationen, Kommunikationsparameter und Varianten. VAG nutzt diese Welt ueber ASAM-Datasets in vielen modernen Steuergeraeten sichtbar. Auch wenn die konkreten Datensaetze nicht frei oeffentlich verteilbar sind, laesst sich ihre Rolle sauber dokumentieren und in eigene Metadatenmodelle ueberfuehren.

Fuer interne Datenbanken hat sich ein dreistufiges Modell bewaehrt: erstens ECU-/Variantenschluessel (Teilenummer, SW-Stand, ASAM-Dateiname, Plattform, Marke, Region, Security-Merkmale), zweitens Diagnoseobjekte (DID, Routine, DTC, Anpassungskanal, Long-Coding-Maske), drittens semantische Annotationen (Beschreibung, Einheit, Wertebereich, Abhaengigkeiten, Preconditions und beobachtete Sessions). So entsteht aus Logdateien, Werkstattwissen und ODX-Hinweisen mit der Zeit ein belastbares Wissenssystem.

6.1 Datenobjekte, die in einer Entwicklerdatenbank typischerweise vorkommen

DIDs fuer Identifikation, Varianten- und Softwarestaende; DTC-Tabellen inklusive Freeze-Frame-Hinweisen; Coding-Masken mit Byte-/Bitbeschreibung; Anpassungskanaele mit Datentyp und Wertebereich; Routinen mit Vorbedingungen; sowie Bus- und Security-Metadaten. Wer nur 'Codingwerte' sammelt, verpasst den groessten Teil der eigentlichen Diagnosearchitektur.

7. MQB-Steuergeraete-Map und Adressraum

Eine universelle Liste aller in allen MQB-Fahrzeugen existierenden Steuergeraete gibt es in dieser Form nicht. Die reale Bestueckung haengt von Marke, Fahrzeugklasse, Baujahr, Assistenzumfang, Motorisierung, Markt und Ausstattungslogik ab. Trotzdem gibt es einen stabilen Kern haeufig wiederkehrender Diagnoseadressen und Funktionsbereiche, auf dessen Basis Entwickler ihre Datenbank strukturieren koennen.

Adresse	Modul	Domaene	Protokoll-/Generationshinweis	Kurzrolle
01	Motorsteuergeraet	Antrieb	UDS/KWP je nach Generation	Motor, Drehmoment, Kennfelder, OBD
02	Getriebesteuergeraet	Antrieb	UDS/KWP	Schaltlogik, Temperatur, Kupplungen
03	ABS/ESP	Fahrdynamik	UDS	Bremsregelung, ESC, Sensorfusion mit Assistenzsystemen
05	Zugangs- und Startberechtigung	Body	UDS	Keyless, Lenkschloss, Freigaben
08	Klimaanlage / HVAC	Body	UDS	Klimabedienung, Luftklappen, Sensorik
09	Zentralelektrik / BCM	Body	UDS	Licht, Komfort, Lastmanagement, viele Anpassungen
0A	Hybrid-/EV-nahe Teilfunktionen	Antrieb	variiert	Fahrzeugspezifisch, nicht in jedem Fahrzeug vorhanden
10	Parklenkassistent	ADAS	UDS	PDC, Einparkhilfe, Ultraschall, Assistenzintegration
13	ACC / Distanzregelung	ADAS	UDS	Radar, Längsregelung, Abstandslogik
14	Daempferregelung	Fahrwerk	UDS	Adaptive Daempfer, Fahrprofil
15	Airbag	Sicherheit	UDS	Crashsensorik, Gurtstraffer, Rückhaltesysteme
16	Lenksaeulenelektronik	Body	UDS	Tasten, Schleifring, Lenkrad-Peripherie
17	Kombiinstrument	HMI	UDS	Anzeigen, Warnungen, Gateway-nahe Anzeige-Funktion
19	CAN-Gateway / Gateway-Host	Architektur	UDS	Bus-Routing, Installation List, Diagnosezugang
1B	Aktive Lenkung / spezielle Fahrwerksfunktionen	Fahrwerk	variiert	Nur in bestimmten Baureihen
20	High Beam Assist / Lichtassistent	ADAS	UDS	Lichtlogik, Kameraabhaengigkeit
23	Bremskraftverstaerker / iBooster	Fahrdynamik	UDS	Elektromechanische Bremsfunktionen
2B	Lenksaeulenverriegelung	Body	UDS	Elektronisches Lenkschloss
32	Sperrdifferenzial / VAQ	Fahrdynamik	UDS	Querdynamik, Antriebsverteilung
36	Sitzverstellung Fahrer	Body	UDS/LIN angebunden	Memory, Spiegelkopplung
39	Sitzverstellung Beifahrer	Body	UDS/LIN angebunden	Memory, Komfort

Adresse	Modul	Domaene	Protokoll-/Generationshinweis	Kurzrolle
3C	Spurwechsel-/Lane-Assist Zusatzmodul	ADAS	UDS	Je nach Baureihe Seitenradar / Assistenten
42	Tuersteuergeraet Fahrer	Body	UDS/LIN	Fensterheber, Spiegel, Verriegelung
44	Lenkhilfe / EPS	Fahrdynamik	UDS	Lenkmoment, Assistenzmoment, EPS
46	Komfortsystem	Body	UDS/KWP alt	Teilweise im BCM integriert oder entfallen
47	Soundsystem	Infotainment	UDS/MOST/CAN	Verstaerker, Audio-Topologie
4B	Multifunktionslenkrad	HMI	UDS/LIN	Tastenmatrix, Heizung, Touch-Bedienfelder je nach Generation
52	Tuersteuergeraet Beifahrer	Body	UDS/LIN	Fensterheber, Spiegel, Verriegelung
55	Leuchtweitenregelung	Body	UDS	Scheinwerfer, Matrix-/AFS-nahe Steuerung
5F	Informationselektronik / Infotainment	Infotainment	UDS	MIB/MMI, Navigation, Car-Menues
61	Batterieregelung	Energie	UDS	12-V-Batterie, Ladezustand, Energiemanagement
65	Reifendruckkontrolle	Fahrdynamik	UDS	Direkt-/Indirektsysteme je nach Plattform
69	Anhaenger	Body	UDS	Anhaengererkennung, Lichtfunktionen
6C	Rueckfahrkamera	Infotainment/ADAS	UDS	Kameraparameter, Bildpfade, Kalibrierung
6D	Kofferraum / Heckklappe	Body	UDS	Elektrische Heckklappe
72	Tuer hinten rechts	Body	UDS/LIN	Komfortfunktionen
75	Telematik	Connectivity	UDS	eCall, Online-Dienste, Backend-Konnektivitaet
77	Telefon	Infotainment	UDS	Freisprechen, Modem/Telefonie je nach Generation
81	Gangwahlelektronik	Antrieb	UDS	Wahlhebel / Shift-by-wire Funktionen
82	Head-up-Display	HMI	UDS	Projektionssystem
8B	Adaptives Licht / AFS	Body	UDS	Kurvenlicht, Matrixfunktionen je nach Fahrzeug
8C	Hybrid-/Hochvoltbatterie	Energie	UDS	BEV/PHEV-spezifisch
8D	Batterieladegeraet / DC-DC	Energie	UDS	Lade- und Wandlungsfunktionen
8E	Front-/Umfeldkamera Assistenz	ADAS	UDS	Kamera-basierte Assistenzfunktionen
A5	Frontsensorik Fahrerassistenz	ADAS	UDS	Lane Assist, VZE, Notbremsanteile je nach Baureihe
A9	Externer Sound / AVAS	E-Mobilitaet	UDS	Acoustic Vehicle Alerting System

Adresse	Modul	Domaene	Protokoll-/Generationshinweis	Kurzrolle
B7	Schnittstellensteuerung EV	E-Mobilitaet	UDS	Ladebuchse, Verriegelung, Kommunikation
BB	Tuer hinten links	Body	UDS/LIN	Komfortfunktionen; teilweise andere Adresskonventionen je Modell
C6	Batteriemanagement HV	Energie	UDS	Plattformabhaengig, speziell MEB/PHEV
D6	Ladesystem HV	Energie	UDS	AC/DC-Ladesteuerung, Ladefreigaben

Wichtig ist die Trennung zwischen 'Adresse wird theoretisch unterstuetzt' und 'Adresse ist im konkreten Fahrzeug bestueckt'. Modulfinder und fahrzeugspezifische Auto-Scans zeigen genau diese Logik: Nicht jede moegliche Adresse ist in jedem Auto aktiv. Ein gutes Tool praesentiert deshalb immer sowohl den theoretischen Adressraum als auch den im Scan beobachteten Steuergeraetesatz.

8. Gateway, Routing und Bus-Topologie

Das Gateway ist fuer Entwickler das organisatorische Zentrum der Diagnoselandschaft. Es entscheidet, in welche Teilnetze Anfragen weitergereicht werden, kennt die Installation List und ist oft der erste Ort, an dem Plattform- und Security-Unterschiede sichtbar werden. In aelteren Fahrzeugen ist die Funktion vor allem Routing und Netzkoordination. In neueren Fahrzeugen kommen Filter, Restriktionen und Host-Funktionen hinzu.

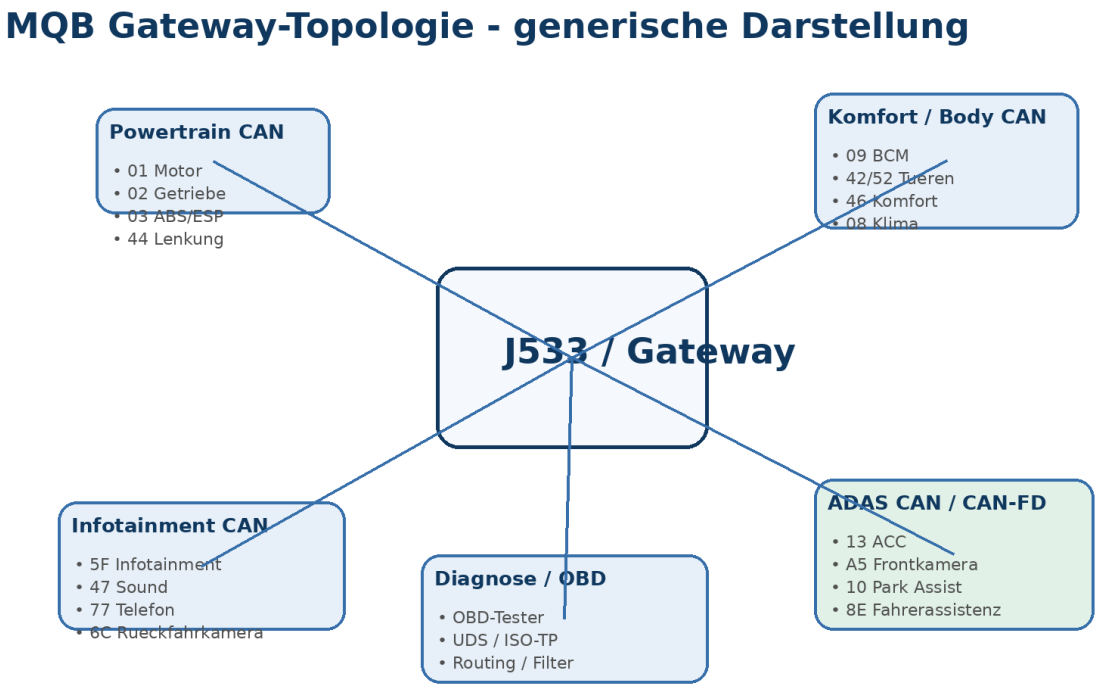


Abbildung 2: Generische MQB-Topologie mit Gateway als zentralem Diagnose- und Routing-Knoten.

Fuer Logging und Toolentwicklung ist wichtig: Ein am OBD-Port beobachteter Verkehr ist nicht deckungsgleich mit dem Rohverkehr auf einem internen Teilbus. Das Gateway kann Nachrichten umsetzen, filtern oder nur zustandsabhaengig weiterleiten. Wer Buskarten erstellt, sollte deshalb stets trennen zwischen physischem Teilbus, diagnostischem Pfad und tatsaechlich beobachtbarer OBD-Perspektive.

8.1 Installation List und Feature-Erkennung

Die Installation List ist in der Praxis ein Schluesselobjekt fuer automatische Fahrzeugprofilierung. Sie erlaubt, beobachtete Funktionen gegen erwartete Domaenen zu plausibilisieren, fehlende Module zu erkennen und viele Logging- oder UI-Elemente dynamisch einzublenden. In einem Entwicklerwerkzeug kann sie als Startpunkt fuer modulbezogene Workflows dienen: beispielsweise nur die Menues fuer ADAS anzeigen, wenn entsprechende Adressen oder Datensaetze vorhanden sind.

9. UDS-Services fuer VAG-Entwickler

Die folgende Tabelle listet die wichtigsten UDS-Services aus Sicht von Diagnose- und Toolentwicklern. Die reine Existenz eines Service bedeutet noch nicht, dass er in jeder Session oder auf jedem Steuergeraet verfuegbar ist. Genau hier greifen Session-Konzept, Security-Level, Preconditions und OEM-spezifische ODX-Definitionen.

SID	Name	Typischer Zweck	Hinweise
0x10	Diagnostic Session Control	Wechsel in Default-, Extended- oder Programming-Session	Basis fuer viele Folgefunktionen
0x11	ECU Reset	Neustart des Steuergeraets	Nicht in allen Sessions erlaubt
0x14	Clear Diagnostic Information	DTCs loeschen	Kann Preconditions haben
0x19	Read DTC Information	DTCs und Status lesen	Hauefig kombiniert mit Freeze-Frame-Kontext
0x22	Read Data By Identifier	DIDs lesen	Wichtigster Service fuer Identifikation und Messwerte
0x23	Read Memory By Address	Direktes Lesen adressierter Speicherbereiche	Stark eingeschraenkt, OEM- und Security-abhaengig
0x24	Read Scaling Data By Identifier	Skalierungsdaten lesen	Selten im Alltags-Tooling genutzt
0x27	Security Access	Authentifizierter Zugriff auf Funktionsgruppen	Nur abstrakt dokumentiert
0x28	Communication Control	Kommunikation aktiv/inaktiv schalten	Test-/Diagnosekontexte
0x2A	Read Data By Periodic Identifier	Periodische Datenuebertragung	Nicht ueberall implementiert
0x2C	Dynamically Define Data Identifier	Temporare Datenobjekte definieren	Fortgeschrittene Diagnosefaelle
0x2E	Write Data By Identifier	DIDs schreiben	Session, Preconditions und Security kritisch
0x2F	Input Output Control By Identifier	Direkte I/O-Ansteuerung	Vor allem Test-/Servicekontexte
0x31	Routine Control	Kalibrierungen, Lernwerte, Tests, Service-Routinen	Sehr haeufig in Werkstattablaeufen
0x34	Request Download	Flash-/Programmierpfad starten	Nur in passenden Sessions
0x35	Request Upload	Daten aus ECU anfordern	OEM- und Security-abhängig
0x36	Transfer Data	Datenblocktransfer	Mit 0x34/0x35 gekoppelt

SID	Name	Typischer Zweck	Hinweise
0x37	Request Transfer Exit	Transfer sauber beenden	Programmierkontext
0x3E	Tester Present	Session am Leben halten	Fuer laengere Routinen oft essenziell
0x83	Access Timing Parameter	Timing-Parameter lesen/setzen	Nicht alltaeglich, aber standardisiert
0x84	Secured Data Transmission	Gesicherte Uebertragung	In der Praxis OEM-spezifisch relevant
0x85	Control DTC Setting	DTC-Setzung aktivieren/deaktivieren	Vor allem Entwicklungs-/Testfaelle
0x86	Response On Event	Ereignisgetriebene Antworten	Selten im normalen Werkstattkontext
0x87	Link Control	Kommunikationsparameter aendern	Spezialfaelle

9.1 Negative Response Codes (NRC)

Wer UDS-Tooling baut, muss NRCs explizit modellieren. In der Praxis sind sie kein Stoergeraeusch, sondern wesentlicher Teil des Diagnosezustands: ConditionsNotCorrect, RequestOutOfRange oder SecurityAccessDenied transportieren fuer den Entwickler oft mehr Information als ein pauschales 'fehlgeschlagen'. Ebenso wichtig ist die Behandlung von ResponsePending und TesterPresent, damit laengere Routinen oder traege Sessions stabil bleiben.

Code	NRC	Bedeutung	Developer-Hinweis
0x10	GeneralReject	Generische Ablehnung	Nur als letzter Rueckfall sinnvoll interpretieren
0x11	ServiceNotSupported	Service nicht verfuegbar	Oft Session- oder ECU-spezifisch
0x12	SubFunctionNotSupported	Subfunktion nicht verfuegbar	Parameter genauer pruefen
0x13	IncorrectMessageLengthOrInvalidFormat	Format oder Laenge falsch	Parser, DLC und Parameteraufbau pruefen
0x21	BusyRepeatRequest	Steuergeraet beschaeftigt	Retry-Strategie noetig
0x22	ConditionsNotCorrect	Vorbedingungen nicht erfuellt	Zuendung, Fahrzeugzustand, Session, Geschwindigkeit usw.
0x24	RequestSequenceError	Ablauffehler	Typisch bei Security oder Routine-Control
0x31	RequestOutOfRange	Parameter ausserhalb gueltigem Bereich	Falscher DID, Subfunktion oder Wert

Code	NRC	Bedeutung	Developer-Hinweis
0x33	SecurityAccessDenied	Kein Zugriff auf diese Funktionsgruppe	Capability statt Fehler modellieren
0x35	InvalidKey	Security-Zugang fehlgeschlagen	Nicht weiter vertiefen
0x36	ExceededNumberOfAttempts	Zu viele Fehlversuche	Backoff und Sperrzeiten beachten
0x37	RequiredTimeDelayNotExpired	Wartezeit noch nicht abgelaufen	Timing korrekt einbauen
0x78	ResponsePending	Antwort folgt spaeter	Timeout-Modell sauber implementieren
0x7E	SubFunctionNotSupportedInActiveSession	Subfunktion in dieser Session nicht erlaubt	Sessionmanagement pruefen
0x7F	ServiceNotSupportedInActiveSession	Service in dieser Session nicht erlaubt	Sessionmanagement pruefen

10. DIDs: Was standardisiert ist und was variantenabhaengig bleibt

Ein typischer Wunsch in Projekten ist eine 'vollstaendige DID-Datenbank mit 1000+ IDs'. Die ehrliche Entwicklerantwort lautet: Einen solchen universell korrekten, oeffentlichen und markenuebergreifenden VAG-Katalog gibt es in dieser Form nicht. Viele Identifier sind ECU- und Datensatz-spezifisch, manche standardisiert, viele OEM-spezifisch. Ein serioeses Handbuch darf hier nicht mit erfundenen Inhalten arbeiten.

Was sich sehr wohl dokumentieren laesst, sind standardisierte oder in der Praxis breit beobachtete Identifikations- und Metadaten-DIDs sowie ein belastbares Datenbankschema, um oeffentliche und interne Erkenntnisse zusammenzufuehren. Genau das macht die folgende Uebersicht.

DID	Bezeichnung	Typische Nutzung	Hinweis
0xF190	VIN	Fahrzeugidentifikation	Breit standardisierter Identifikator
0xF187	Fahrzeug-/ECU-Teilenummer	Identifikation	OEM-Darstellung kann variieren
0xF189	Hardware-Nummer	Identifikation	Hilfreich fuer Variantenzuordnung
0xF18A	Software-Nummer	Identifikation	Mit Softwarestand korrelieren
0xF18C	ECU-Seriennummer	Identifikation	Nicht immer offen lesbar
0xF18D	Systemlieferanten-ID	Identifikation	Lieferantenbezug
0xF18E	Herstellungsdatum / ECU-Date	Identifikation	Interpretation je Datensatz
0xF180	Boot-Software-ID	Programmier-/Ident-Kontext	Nicht auf jedem ECU sichtbar
0xF181	Application-Software-ID	Softwareidentifikation	Typischer Build-/SW-Anchor
0xF182	Application-Daten-ID	Variantendaten	Datensatznahe Hinweise moeglich
0xF183	Boot-Software-Fingerprint	Programmierhistorie	Nicht universell
0xF184	Application-Software-Fingerprint	Programmierhistorie	Nicht universell
0xF186	Active Diagnostic Session	Diagnosekontext	Nuetzlich fuer Debugging
0xF18B	ECU-Hardware-Nummer	Identifikation	Oft in OEM-Tools benoetigt
0xF197	Systemname / ECU-Name	Identifikation	Kann variieren

10.1 Empfohlenes DID-Datenbankschema

Empfohlene Schluesselfelder: ECU-Familie, ASAM-Datensatz, DID-Hexwert, Name, Datentyp, Endianness, Einheit, Wertebereich, Session-Anforderungen, Security-Level, Preconditions, bekannte Softwarestaende, Notizen und mindestens ein Feld fuer Vertraulichkeit bzw. Quellenqualitaet. Ohne Quellenqualitaet vermischt man

schnell belastbare Erkenntnisse mit Forumwissen oder unvollstaendigen Mitschnitten.

10.2 OBD-PIDs vs. UDS-DIDs

Im Alltag werden OBD-PIDs und UDS-DIDs oft durcheinandergebracht. OBD-Mode-01-PIDs sind standardisierte emissionsnahe Diagnosedaten. UDS-DIDs sind dagegen Teil der herstellerspezifischen Diagnosedatenmodellierung und koennen alles von Identifikation bis Kalibrierstatus enthalten. Fuer Multi-Tool-Software ist eine saubere Trennung in der UI und in der API unverzichtbar.

PID	Name	Einheit / Typ	Kommentar
0x0C	Engine RPM	1/min	Klassischer OBD-Livewert
0x0D	Vehicle Speed	km/h	Emissions-/OBD-Welt, nicht identisch mit allen OEM-Geschwindigkeiten
0x05	Coolant Temperature	deg C	Nur emissionsnaher Standardwert
0x11	Throttle Position	%	Nicht identisch mit allen internen Fahrpedal-/Klappenwerten
0x0F	Intake Air Temperature	deg C	OBD-Standard
0x10	MAF Air Flow Rate	g/s	Verbrennerlastnahe Standardgroesse
0x2F	Fuel Level Input	%	Je nach Fahrzeug nur eingeschraenkt relevant
0x46	Ambient Air Temperature	deg C	Nur wenn implementiert
0x5C	Engine Oil Temperature	deg C	Nicht in jedem OBD-Stack verfuegbar
0x42	Control Module Voltage	V	Praktisch fuer Bordnetz-Basischecks

11. Coding, Anpassung und Variantendefinition

Unter 'Coding' fallen bei VAG mehrere unterschiedliche Konzepte, die sauber getrennt werden sollten: klassisches Short Coding, Long Coding mit Byte-/Bitmasken, Anpassungskanaele (Adaptation), parametrische Datensaetze, Basic Settings sowie teilweise softwareseitige FEC-/Funktionsfreischaltungen. Viele Missverstaendnisse entstehen, wenn all diese Dinge in einem einzigen Feld namens 'Coding' abgelegt werden.

11.1 Short Coding

Short Coding ist typisch fuer aeltere Generationen und kondensiert mehrere Konfigurationsentscheidungen in kompakten numerischen oder hexadezimalen Strings. Aus Datenbanksicht ist wichtig, dass die semantische Zerlegung oft schlecht dokumentiert ist und stark vom konkreten Steuergeraet abhaengt.

11.2 Long Coding

Long Coding ist fuer moderne VAG-Werkstattpraxis zentral, weil sich Features als Byte-/Bitdefinitionen abbilden lassen. Technisch sollte man dabei immer zwischen physischem Speicherabbild, vom Tool gezeigter Bitmaske und semantischer Beschreibung unterscheiden. Ein gutes internes Modell hat deshalb Felder fuer Byte, Bit, Bitlaenge, Datentyp, Inversion, Abhaengigkeiten und Sichtbarkeit nach Ausstattung.

11.3 Anpassungskanaele

Adaptation ist nicht einfach 'anderes Coding', sondern eher eine parametrisierte Konfigurationsoberflaeche mit Namen, Datentyp, Einheit, Bereich, oft auch Benutzertexten. Gerade im BCM- und Lichtumfeld sind hunderte Kanaele moeglich. Damit ist Adaptation fuer Datenbanken oft wichtiger als Long Coding, weil hier die semantische Benennung vergleichsweise reichhaltig ist.

Kategorie	Technische Form	Typische Inhalte	Metadaten, die man speichern sollte
Short Coding	Kompakter Zahl-/Hexwert	Grundkonfiguration aelterer ECUs	Laenge, Anzeigeformat, Zerlegungsregel
Long Coding	Byte-/Bitmaske	Ausstattungsflags, Funktionsbits	Byte, Bit, Bitlaenge, Abhaengigkeiten, Default
Adaptation	Benannter Kanal	Parameter, Schwellwerte, Modi	Name, Typ, Einheit, Wertebereich, Session, Writeability
Basic Settings	Routine / Prozedur	Anlernen, Kalibrieren, Reset	Vorbedingungen, Ergebnisstatus, Dauer
Datensatz/Parameter	Strukturierte Variantendaten	Kamera-, Radar-, Ladesystem- oder HMI-Parameter	Version, Herkunft, Mapping
FEC/FSC/Funktionsfreigabe	Lizenz-/Freigabelogik	Navigations-/Featurefreigabe	Freigabeart, Abhaengigkeiten, Security

Wichtige Abgrenzung

Das gleiche Fahrzeugfeature kann in Generation A als Long Coding, in Generation B als Anpassungskanal und in Generation C ueber Datensatz oder signierte Freischaltung realisiert sein. Historische Listen ohne Plattform- und ECU-Bezug sind daher als Wissensquelle nur begrenzt belastbar.

12. Oeffentlich sauber dokumentierbare Coding-Beispiele

Eine komplette '1000+ Codings'-Datenbank fuer alle Marken und Baujahre ist ohne OEM-Daten und fahrzeugspezifische Datensaetze nicht serioes veroeffentlichbar. Was sich sinnvoll dokumentieren laesst, sind wiederkehrende Kategorien und einige oeffentlich breit bekannte Beispieltypen. Genau diese sollten in internen Tools als Kategoriesystem und nicht als wahllose Liste einzelner Tweaks modelliert werden.

Bereich	Typischer Trager	Oeffentlich oft diskutierte Beispielklasse	Entwicklerischer Blick
Licht / Komfort	Adresse 09 BCM	Komfortblinker, Lichtlogik, Tagfahrlichtvarianten	Stark adaptionsgetrieben, hohe Kanalanzahl
Zugang / Verriegelung	05 / 46 / BCM	Auto-Lock, Spiegelanklappung, Komfortoeffnen	Plattformabhaengig verteilt
Infotainment / 5F	5F Informationselektronik	Menueoptionen, HMI-Verhalten, Audiooptionen	Grenze zwischen Anzeige und Fremdsteuergeraeten beachten
Kombiinstrument	17	Anzeigeoptionen, Darstellungen, Hinweise	Cluster oft nur HMI fuer Fremddaten
Fahrerassistenz	13 / A5 / 8E / 10	Assistenzanzeigen, Sensorparameter, Betriebsmodi	Security und Kalibrierungslogik zentral
Klimatisierung	08	Komfort- und Menueparameter	Haeufig tiefe LIN/Body-Verknuepfung
Anhaenger	69	Anhaengererkennung, Lichtlogik	Abhaengig von Gateway und BCM

Fuer Entwickler ist die eigentliche Mehrarbeit nicht das Sammeln einzelner Eintraege, sondern die Versionierung: Welcher Eintrag wurde an welchem Fahrzeug, mit welchem Datensatz, welcher Session, welchem Security-Level, welcher Werkzeugversion und welchem Ergebnis beobachtet? Wer das sauber erfasst, hat nach einiger Zeit eine weitaus wertvollere Datenbank als jede ungepflegte Copy-Paste-Sammlung.

13. MIB / Infotainment-Architektur

MIB steht fuer 'Modularer Infotainment Baukasten'. Entwicklerrelevant ist weniger die Marketingbezeichnung als die Trennung von Anzeige, Head Unit, Audio, Telefonie/Telematik, Smartphone-Integration und Fahrzeugintegration. Im MQB-Kontext ist die J794-Logik bzw. Informationselektronik ein zentraler Baustein. Je nach Generation und Markt koennen Aufbau, Rechenleistung und Komponentenplatzierung variieren.

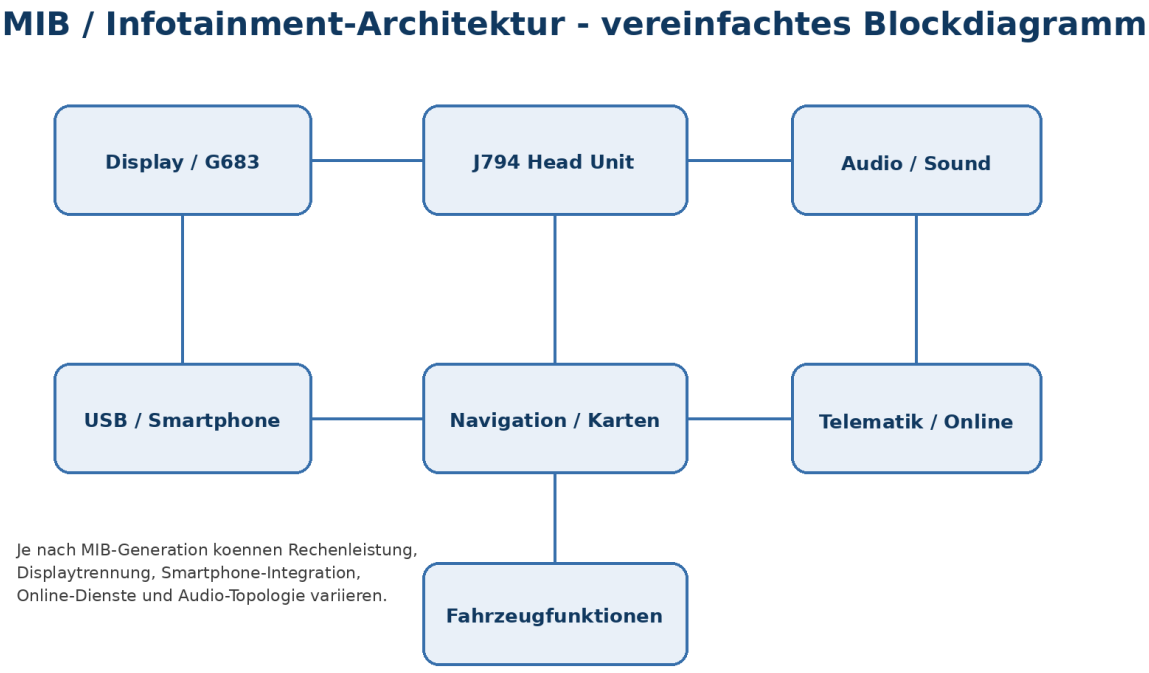


Abbildung 3: Vereinfachtes MIB-Blockdiagramm. Die konkrete Ausfuehrung variiert zwischen MIB I, MIB II, MIB3 und markenspezifischen Systemen.

Fuer Diagnose- und Datenbankzwecke sollte die Infotainment-Domaene mindestens in folgende Funktionsebenen zerlegt werden: Anzeige/HMI, Head Unit, Audio-Pfad, Navigation/Content, Connectivity/Backend sowie Fahrzeugfunktionen (Car-Menues, Fahrprofile, Assistenzanzeigen). Erst diese funktionale Trennung ermoeglicht nachvollziehbare Fehlersuche und saubere Modellierung von Datenquellen.

13.1 Wichtige Entwicklerfragen in der MIB-Domaene

Welche Funktionen liegen im Infotainment selbst und welche in vorgelagerten Steuergeraeten? Welche Daten kommen vom Gateway, vom Kombiinstrument oder von ADAS-Steuergeraeten? Welche Menues sind reine Anzeige von Fremddaten und welche Aenderungen werden als Diagnose-/Codingvorgang an andere Steuergeraete delegiert? Solche Fragen entscheiden ueber die richtige Architektur eines Diagnose-Frontends.

14. ADAS-Modulstruktur

ADAS in VAG-Fahrzeugen ist kein einzelnes Steuergeraet, sondern ein Verbund aus Sensorik, Plausibilisierung, HMI und Aktorik. Radar, Frontkamera, Park-/Ultraschallsysteme, teils Rundumsicht- oder Zusatzkameras, Lenkung und Bremsen bilden zusammen die Assistenzfunktion. In neueren Architekturen steigen Rechenleistung, Filterung und Security-Anforderungen deutlich.

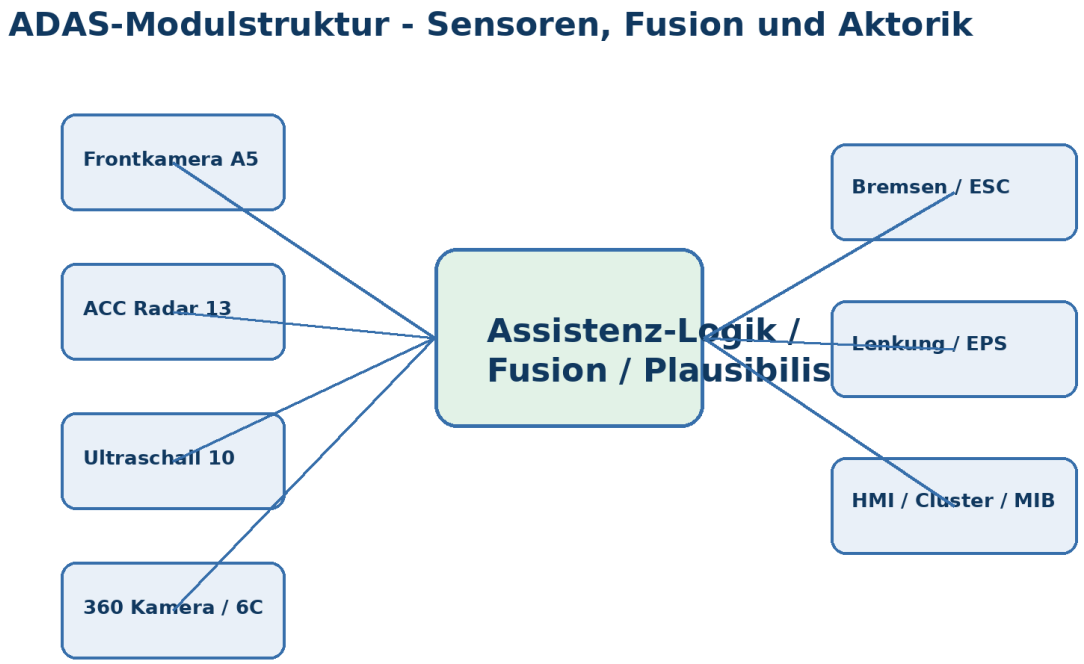


Abbildung 4: Generische ADAS-Struktur mit Sensorik, Fusion/Logik und Aktorik.

Entwickler sollten ADAS-Daten nie isoliert pro Sensor betrachten. Entscheidend ist die Verkettung von Rohsensorik, fusionierter Zustandslogik, Freigabebedingungen, HMI-Hinweisen und Aktorikgrenzen. In einem technischen Datenmodell brauchen ADAS-Funktionen daher Querverweise zwischen Kamera, Radar, Bremse, Lenkung, Cluster und HMI.

Modul/Domaene	Beispieladresse	Rolle	Entwicklerhinweis
ACC Radar	13	Abstandsmessung und Laengsregelungsanteile	Mit Bremse, Motor und HMI verknuepft
Frontkamera	A5 / 8E	Spur, Schilder, teils Notbremsanteile	Kalibrierung und Sichtbedingungen relevant
Park Assist / Ultraschall	10	Nahbereich, Einpark- und Warnfunktionen	HMI- und Gateway-nahe Integration
Rueckfahr-/Umfeldkamera	6C	Bildpfad und Kalibrierung	Teilweise Infotainment-naher Datenpfad
EPS / Lenkung	44	Aktuatorik fuer Spur-/Lenkfunktionen	Sicherheitskritische Freigaben
ABS/ESC	03	Bremseneingriffe und Plausibilisierung	Zentrale Sicherheits-/Dynamikrolle

Modul/Domaene	Beispieladresse	Rolle	Entwicklerhinweis
Kombiinstrument / HMI	17 / 5F	Warnungen, Assistentenanzeige	Nur Anzeigeebene, nicht Assistenzlogik selbst

15. Security, SFD und Diagnose-Restriktionen

Moderne VAG-Fahrzeuge unterscheiden immer staerker zwischen Lesen, Servicieren und Aendern. Sicherheitsmechanismen wie SFD oder SFD2 fuehren dazu, dass bestimmte schreibende Funktionen nur mit Freigabe, Token oder signierten Nachrichten moeglich sind. Gleichzeitig bleiben viele servicebezogene oder lesende Funktionen weiterhin verfuegbar. Genau diese Trennung muss jedes professionelle Tool transparent kommunizieren.

Aus Entwicklersicht bedeutet das: Capability-Matrizen statt Schwarz-Weiss-Denken. Ein Steuergeraet kann in Session A lesbar sein, in Session B Basic Settings erlauben, aber Coding oder Adaption blockieren. Andere Knoten laufen nur read-only hinter Diagnosefilter oder Firewall. Diese Restriktionen sollten nie als 'Fehler' der ECU modelliert werden, sondern als Architekturzustand.

Funktionsbereich	Typisch frei lesbar	Typisch servicefaehig	Typisch eingeschraenkt
Identifikation	Ja	Ja	Selten
DTC lesen	Ja	Ja	Selten
Messwerte / DIDs lesen	Hauefig	Ja	Je nach Filter/Session eingeschraenkt
Basic Settings	Teilweise	Hauefig werkstattbezogen	Je nach System und Session
Adaptation schreiben	Uneinheitlich	Teilweise	Bei neueren Systemen oft freigabepflichtig
Coding schreiben	Uneinheitlich	Teilweise	Bei neueren Systemen oft SFD/SFD2-relevant
Programmierfunktionen	Nein	OEM-/Werkstattprozess	Stark eingeschraenkt
Assistenz-/Cybersecurity-nahe Aenderungen	Nein	Selten	Oft signatur- oder freigabepflichtig

Worauf in Dokumentationen geachtet werden sollte

Nie 'Bypass' oder Umgehungslogik dokumentieren. Stattdessen festhalten: welcher Funktionsbereich grundsaetzlich eingeschraenkt ist, welche OEM- oder Werkstattprozesse vorgesehen sind und welche Entwicklungs- oder Testfaelle auch ohne schreibenden Zugriff bereits abbildbar sind.

16. Praxisorientierter Entwickler-Workflow

16.1 Fahrzeugprofilierung

Beginne mit der Profilierung des Fahrzeugs: Plattform, Marke, Modelljahr, Gateway-Identitaet, Installation List, beobachtete Adressen, Softwarestaende, ASAM-Datensaetze und Security-Indikatoren. Ohne diesen Schritt erzeugen spaetere Logs nur unverbundene Rohdaten.

16.2 Kommunikationsaufnahme

Validiere physische Verbindung, Versorgung, Zuendungszustand, Session-Eintritt, TesterPresent und Gateway-Sichtbarkeit. Ein grosser Teil vermeintlicher Diagnosefehler ist in Wahrheit Zustandslogik des Fahrzeugs.

16.3 Datenmodell aufbauen

Lege fuer jedes beobachtete Steuergeraet eine Stammdatentabelle an und verknuepfe daran DIDs, DTCs, Routinen, Coding-Objekte, Anpassungen und erkannte Buszuordnung. Kennzeichne Quelle und Vertrauensgrad. Gerade bei heterogenen Quellen ist ein Feld fuer 'beobachtet', 'abgeleitet', 'aus Dokumentation' oder 'unsicher' enorm wertvoll.

16.4 UI und Tooling

Ein gutes Tool fuehrt den Nutzer nicht ueber rohe Hex-Frames, sondern ueber Ebenen: Fahrzeug -> Domaene -> Steuergeraet -> Funktion -> Datenobjekt. Rohframes bleiben trotzdem erreichbar, idealerweise mit Frame-, Session- und Decoding-Kontext. So entstehen Werkzeuge, die sowohl Entwickler als auch fortgeschrittene Diagnoseanwender produktiv nutzen koennen.

17. Referenzarchitektur fuer eine interne Wissensdatenbank

Die produktivste Form, VAG-Wissen zu organisieren, ist selten ein PDF allein, sondern eine Datenbank plus Dokumentation. Das PDF dient als Referenz, die Datenbank als auswertbarer Wissensspeicher. Das Schema unten zeigt eine moegliche Minimalarchitektur.

Tabelle	Zweck	Pflichtfelder
vehicle	Fahrzeugstammdaten	VIN, Marke, Modell, Plattform, Modelljahr, Region
ecu_instance	Beobachtetes Steuergeraet im konkreten Fahrzeug	vehicle_id, adresse, name, sw_nummer, hw_nummer, asam, security_status
ecu_family	Logische ECU-Familie ueber Fahrzeuge hinweg	familienname, lieferant, domain, protokoll
did_definition	DID-Metadaten	ecu_family_id, did_hex, name, datentyp, einheit, quelle, vertrauensgrad
dtc_definition	Fehlercode-Metadaten	ecu_family_id, dtc, text, statusmodell
coding_item	Long-/Short-Coding-Element	ecu_family_id, coding_typ, byte, bit, name, default, abhaengigkeiten
adaptation_item	Anpassungskanal	ecu_family_id, kanalname, datentyp, min, max, einheit, writeability
routine_item	Routine-Control-Eintrag	ecu_family_id, routine_id, name, preconditions, expected_result
scan_log	Auto-Scan oder Sitzungsmetadaten	vehicle_id, datum, tool, raw_text, parser_version
trace_meta	CAN-/UDS-Mitschnitt-Metadaten	vehicle_id, bus, startzeit, format, kommentar

Zusaetzlich empfehlenswert: Tabellen fuer Fahrzeug-Samples, Scan-Dateien, Mitschnitt-Metadaten, Units, Enumerationen, Security-Capabilities, Build-Historien und Bild-/Topologieanhaenge. Je sauberer die Metadaten modelliert sind, desto weniger Wissen bleibt als unstrukturierter Text in Foren oder Freitextnotizen liegen.

18. Grenzen oeffentlicher Dokumentation

VAG-spezifische Vollstaendigkeit ist oeffentlich nur begrenzt erreichbar. ODX-Datensaetze, markenspezifische DIDs, signierte Freigaben, parametrische Datensaetze und laenderspezifische Varianten sind in weiten Teilen proprietaer oder nur im OEM- oder Werkstattkontext sauber dokumentiert. Ein serioeses Entwicklerhandbuch muss diese Grenze benennen, statt sie mit erfundenen Tabellen zu ueberspielen.

Die praktisch richtige Konsequenz lautet daher: oeffentlich verifizierbare Grundlagen breit und sauber dokumentieren; fahrzeugspezifische Erkenntnisse systematisch als eigene Datenbasis sammeln; und in allen Tabellen klar markieren, ob ein Eintrag standardisiert, oeffentlich belegt, feldbeobachtet oder nur hypothesenbasiert ist.

Anhang A: Glossar

Begriff	Kurzdefinition
ADAS	Advanced Driver Assistance Systems; Fahrerassistenzfunktionen und ihre Sensor-/Aktorikverbuende
ASAM	Standardisierungsorganisation; relevant u. a. fuer ODX (MCD-2 D)
BCM	Body Control Module; typischerweise Adresse 09 in vielen MQB-Fahrzeugen
CAN	Controller Area Network; zentrales Fahrzeugbus-System
CAN-FD	Erweiterung von CAN mit hoeherer Datenrate und groesserer Payload
DID	Data Identifier in UDS
DTC	Diagnostic Trouble Code
DoIP	Diagnostics over IP; Diagnose ueber IP/Ethernet
Gateway	Zentraler Routing- und Diagnoseknoten, oft Adresse 19
KWP	Keyword Protocol, fruehere Diagnosestandards
LIN	Local Interconnect Network; kostenguenstige Peripherieanbindung
MEB	Modularer E-Antriebs-Baukasten
MIB	Modularer Infotainment Baukasten
MQB	Modularer Querbaukasten
NRC	Negative Response Code in UDS
ODX	Open Diagnostic Data Exchange, ASAM MCD-2 D
OBD	On-Board Diagnostics, emissionsnahe Standarddiagnose
SFD	Schutz der Fahrzeugdiagnose
UDS	Unified Diagnostic Services nach ISO 14229

Anhang B: Literatur und Quellenhinweise

Die folgende Liste nennt die wichtigsten oeffentlichen Quellen, auf denen dieses Handbuch inhaltlich basiert. Sie ersetzt keine OEM-Serviceinformation, hilft aber dabei, Architektur und Standards sauber einzuordnen.

- Volkswagen Self-Study Programme 513 'The Golf 2013' und SSP 517 'The Golf 2013 Electrical System' fuer MQB-Grundlagen.

- Volkswagen / NHTSA-Unterlagen zu MIB II fuer die generische MIB-Architektur.

- ASAM MCD-2 D (ODX) Standardbeschreibung fuer das formale Diagnose-Datenmodell.

- Softing UDS- und Standards-Uebersichten fuer UDS, ODX, CAN und DoIP-Kontext.

- Ross-Tech Modulfinder, Diagnostic Procedures und SFD-Dokumentation fuer Diagnoseadressen, typische Modulstrukturen und Restriktionskonzepte.